day2-hadoop-mapreduce

Day 2: HDFS, YARN and MapReduce

Answers to the 15 Day 2 Knowledge Review questions, plus a worked storage calculation and a step-by-step Word Count trace.

Q1. Why does HDFS separate metadata from data storage? What would happen if all metadata were stored on DataNodes?

The NameNode holds only metadata (file names, directory tree, block list, block locations) in memory, while DataNodes hold the actual blocks on disk. This keeps lookups fast, gives one authoritative view of the namespace, and lets bulk data flow directly between clients and DataNodes, so the NameNode never becomes a data-transfer bottleneck.
If metadata lived on DataNodes:

- A client would have to ask many nodes to find where a file's blocks are, which is slow.
- Keeping copies of the namespace consistent across nodes would need constant coordination.
- Losing a DataNode could lose part of the file system map, not just data.
- Data placement and re-replication decisions would have no single coordinator.

Q2. Describe the default block replication placement policy and why it uses multiple racks.

With replication factor 3:

1. First replica on the writer's node (or a random node in the client's rack).
2. Second replica on a node in a different rack.
3. Third replica on another node in the same rack as the second.

Reasons:

- Durability: losing a whole rack (switch or power failure) cannot destroy every copy.
- Performance: two replicas share a rack, so write traffic crosses racks only once, and reads can often stay in-rack.

The trade-off is extra cross-rack bandwidth on writes in exchange for surviving rack failure.

Q3. How does the YARN ResourceManager decide which application gets resources? Discuss FIFO, Capacity and Fair schedulers.

The ResourceManager uses a pluggable scheduler. Applications ask for containers (memory, vCores, locality preference); the scheduler decides who gets what based on its policy.

Scheduler	How it works	Good for	Weakness
FIFO	First submitted, first served	Single-user or test clusters	A big job blocks all later jobs
Capacity	Queues get guaranteed shares (e.g. Finance 40%, ML 60%); idle capacity can be borrowed	Multi-team clusters with guarantees	Needs queue planning
Fair	Resources are shared out so active apps get roughly equal shares over time	Mixed short and long jobs	Long jobs may run slower

Most production clusters use Capacity or Fair, sometimes with preemption to reclaim borrowed resources.

Q4. Distinguish the roles of the ResourceManager, NodeManager and ApplicationMaster.

Component	Count	Role
ResourceManager	One per cluster (plus standby for HA)	Global scheduling and container allocation; tracks cluster capacity; starts each app's first container
NodeManager	One per worker node	Launches and monitors containers on its node, reports health and usage by heartbeat, aggregates logs
ApplicationMaster	One per application	Negotiates containers from the RM, places tasks (with data locality), monitors and retries failed tasks

Short version: the RM decides how much, the AM decides what runs where for its own job, and the NM actually runs it.

Q5. Explain the shuffle-and-sort phase. Why is it usually the most expensive part of a MapReduce job?

After the map phase, all values for the same key must reach the same reducer.

1. Each mapper partitions its output by key (default `hash(key) mod R`), sorts it, and spills it to local disk.
2. Each reducer pulls its partition from every mapper over the network.
3. The reducer merges and sorts the pieces by key.
4. Values for each key are grouped and passed to the reduce function.

It is expensive because it combines disk writes, network transfer from every mapper to every reducer, sorting, and a barrier (reducers cannot finish until all maps finish). Combiners, compression of map output and good partitioning reduce the cost.

Q6. Compare map tasks and reduce tasks across input, output, data movement and fault behaviour.

Aspect	Map task	Reduce task
Input	One input split (usually one HDFS block)	Its partition of intermediate data from all mappers
Output	Intermediate key-value pairs on local disk	Final output written to HDFS
Data movement	Mostly local read (data locality)	Network-heavy shuffle
Number	One per split, decided by input size	Set by the user
Fault behaviour	Rerun on another node; its input is still in HDFS	Rerun and refetch map outputs; may force maps to rerun if their node died

Q7. What distinguishes a combiner from a reducer? Give an example where a combiner would be unsafe.

A combiner is an optional mini-reduce that runs on each mapper's output before the shuffle to cut network traffic. A reducer runs after the shuffle over all values for a key from all mappers.

- The framework may run a combiner zero, one or many times, so the result must not depend on it.
- It is only safe when the operation is commutative and associative (sum, count, max, min).

Unsafe example: average. If mapper A has values 1, 2, 3 (avg 2) and mapper B has 10 (avg 10), averaging the averages gives 6, but the true average is 4. Fix: emit (sum, count) pairs and divide only in the reducer.

Q8. How does HDFS fault tolerance differ from YARN's? Which failures does each handle?

Layer	Protects	Failures handled	Mechanism
HDFS	Data	DataNode crash, disk failure, corrupted block, NameNode failure (with HA)	Block replication, checksums, heartbeats, re-replication, standby NameNode with JournalNodes
YARN	Running work	NodeManager crash, container failure, ApplicationMaster failure, RM failure (with HA)	Heartbeats, container reallocation, AM restart, standby ResourceManager

MapReduce adds a third layer on top: task retry and speculative execution. HDFS keeps the data safe; YARN keeps compute running.

Q9. Contrast reduce-side join and map-side join. When would you choose each?

	Reduce-side join	Map-side (replicated) join
How	Mappers tag each record with its source and emit the join key; matching records meet at a reducer	Small table is loaded into memory on every mapper; large table is joined as it streams through
Shuffle	Full shuffle of both datasets	None for the large table
Works when	Both datasets are large	One dataset fits in memory
Cost	High network and disk	Low, but memory per mapper

Choose map-side when one side is small and stable (e.g. a product lookup table). Choose reduce-side when both sides are large (e.g. orders joined with clickstream).

Q10. Why is YARN's separation of resource management from processing logic important for multi-tenant clusters?

Before YARN (Hadoop 1), the MapReduce JobTracker did both scheduling and resource management, so only MapReduce could run well on the cluster.
With YARN:

- MapReduce, Spark, Tez and others share one cluster and one copy of the data.
- Each application brings its own ApplicationMaster with its own logic.
- Queues and schedulers give teams guaranteed or fair shares.
- Containers isolate memory and CPU so one tenant cannot starve another.
- Utilisation goes up because idle capacity is shared.

Q11. Word Count on 1 TB: one reducer is far slower than the others. Diagnose and give two mitigations.

Cause: data skew. Word frequencies follow a Zipf-like distribution, so a few words ("the", "a", "of") produce a huge share of the pairs. With hash partitioning, every pair for "the" goes to the same reducer, which becomes a straggler.
Mitigations (any two):

- Use a combiner so each mapper sends one (word, partial_count) per word instead of millions of (word, 1).
- Salt hot keys: emit ("the#3", 1) with a random suffix, aggregate per salted key, then sum the partial results in a second step.
- Use a custom partitioner that spreads known hot keys over several reducers.
- Filter stop words if they are not needed in the result.

Speculative execution does not fix skew, because the duplicate task has the same oversized input.

Q12. A job processes many 10 KB files and the NameNode runs into memory problems. Why, and how do you fix it?

Every file, directory and block is an object in NameNode memory (roughly 150 bytes each as a rule of thumb). A 10 KB file still costs at least one file object and one block object, so millions of tiny files consume NameNode heap while storing very little data. It also creates one map task per small file, so task start-up overhead dominates.

Fixes:

- Merge small files into large ones before or after loading (e.g. a compaction job).
- Store them in container formats: SequenceFiles, HAR (Hadoop Archives), or columnar files like Parquet/ORC.
- Use CombineFileInputFormat so one map task reads many small files.
- Fix the producer to write larger batches.

Q13. MapReduce batch jobs and Spark analytics share a cluster. How does YARN keep both fair, and what configuration matters?

Both run as YARN applications with their own ApplicationMasters and request containers from the same ResourceManager. Fairness comes from the scheduler.

Configuration that matters:

- Scheduler type (Capacity or Fair) and queue definitions, e.g. a `batch` queue and an `analytics` queue with guaranteed shares.
- Maximum capacity per queue so one queue cannot take everything.
- User limits within a queue.
- Preemption to reclaim borrowed resources when the owning queue needs them.
- Container sizes (min/max memory and vCores) and NodeManager capacity so containers fit the hardware.
- For Spark, dynamic allocation so idle executors are released back to YARN.

Q14. A map task fails repeatedly on the same node. What does the framework do, and what should an administrator do?

Framework:

- The ApplicationMaster retries the task, normally on a different node (default up to 4 attempts per task).
- If a node causes repeated failures, the AM blacklists it for that job.
- If all attempts fail, the job fails.
- The NodeManager health checker can mark the node unhealthy so the RM stops scheduling there.

Administrator:

- Read container logs (log aggregation, `yarn logs –applicationId ...`).
- Check the node: disk full or failing, bad memory, wrong JDK or library version, misconfiguration.
- Decommission or repair the node, then recommission it.
- Confirm HDFS has re-replicated any blocks from failed disks.

Q15. A data scientist wants interactive queries on the data your MapReduce cluster processes. Is HDFS + MapReduce suitable? What would you recommend?

HDFS is fine as storage, but MapReduce is not suitable for interactive work. Each query becomes a job with container start-up, disk writes between stages and a full shuffle, so latency is minutes, not seconds.

Recommended:

- Spark SQL (in-memory, Catalyst optimizer) or Trino/Presto or Impala for SQL queries in seconds.

- Hive on Tez with LLAP if the team already uses Hive.
- Store curated data as partitioned Parquet or ORC so queries read only needed partitions and columns.
- Pre-aggregate common results into small tables for dashboards.

Keep MapReduce (or Spark batch) for heavy scheduled ETL.

Worked example: block count and raw storage

File size S = 1 GB = 1024 MB, block size B = 128 MB, replication R = 3.

- Blocks: ceil(1024 / 128) = 8 blocks.
- Block replicas stored: 8 x 3 = 24.
- Raw storage: 1024 MB x 3 = 3072 MB = 3 GB.
- NameNode tracks 1 file object + 8 block objects.

Same question for a 300 MB file: ceil(300 / 128) = 3 blocks (128 + 128 + 44 MB). Raw storage is 300 x 3 = 900 MB, not 3 x 128 x 3, because the last block only uses the space it needs.

Worked example: Word Count step by step

Input, two documents (each becomes one split and one map task):

- doc1: `the cat sat on the mat`
- doc2: `the dog sat on the log`

1. Map (doc1): (the,1) (cat,1) (sat,1) (on,1) (the,1) (mat,1)
2. Map (doc2): (the,1) (dog,1) (sat,1) (on,1) (the,1) (log,1)
3. Combine (optional, per mapper):
    - doc1: (the,2) (cat,1) (sat,1) (on,1) (mat,1)
    - doc2: (the,2) (dog,1) (sat,1) (on,1) (log,1)
4. Partition: each key goes to reducer `hash(key) mod R`. With R = 2, say "the", "sat", "mat", "log" go to reducer 0 and the rest to reducer 1.
5. Shuffle and sort: reducer 0 receives, sorted by key: log:[1], mat:[1], sat:[1,1], the:[2,2].
6. Reduce: sum the list for each key.
7. Output (two part files in HDFS):

```
part-r-00000: log 1, mat 1, sat 2, the 4
part-r-00001: cat 1, dog 1, on 2
```

Note the combiner cut the pairs sent over the network from 12 to 10. On real data the saving is much larger.

Quick recall

- NameNode = metadata in memory; DataNodes = blocks on disk.
- Default block size 128 MB, default replication 3.
- Placement: local node, then a remote rack, then another node on that same remote rack.
- Data never flows through the NameNode.
- YARN: RM schedules cluster, NM runs containers, AM runs one application.
- MapReduce flow: split, map, (combine), partition, shuffle and sort, reduce, output.
- Default partitioner: hash(key) mod number of reducers.
- Map output goes to local disk; final output goes to HDFS.
- Combiners only for commutative and associative operations; never average directly.
- Map-side join needs one small table; reduce-side join handles two large tables.

- Small files hurt the NameNode; merge them or use container formats.
- Skew is fixed with combiners, salting or custom partitioners, not speculative execution.